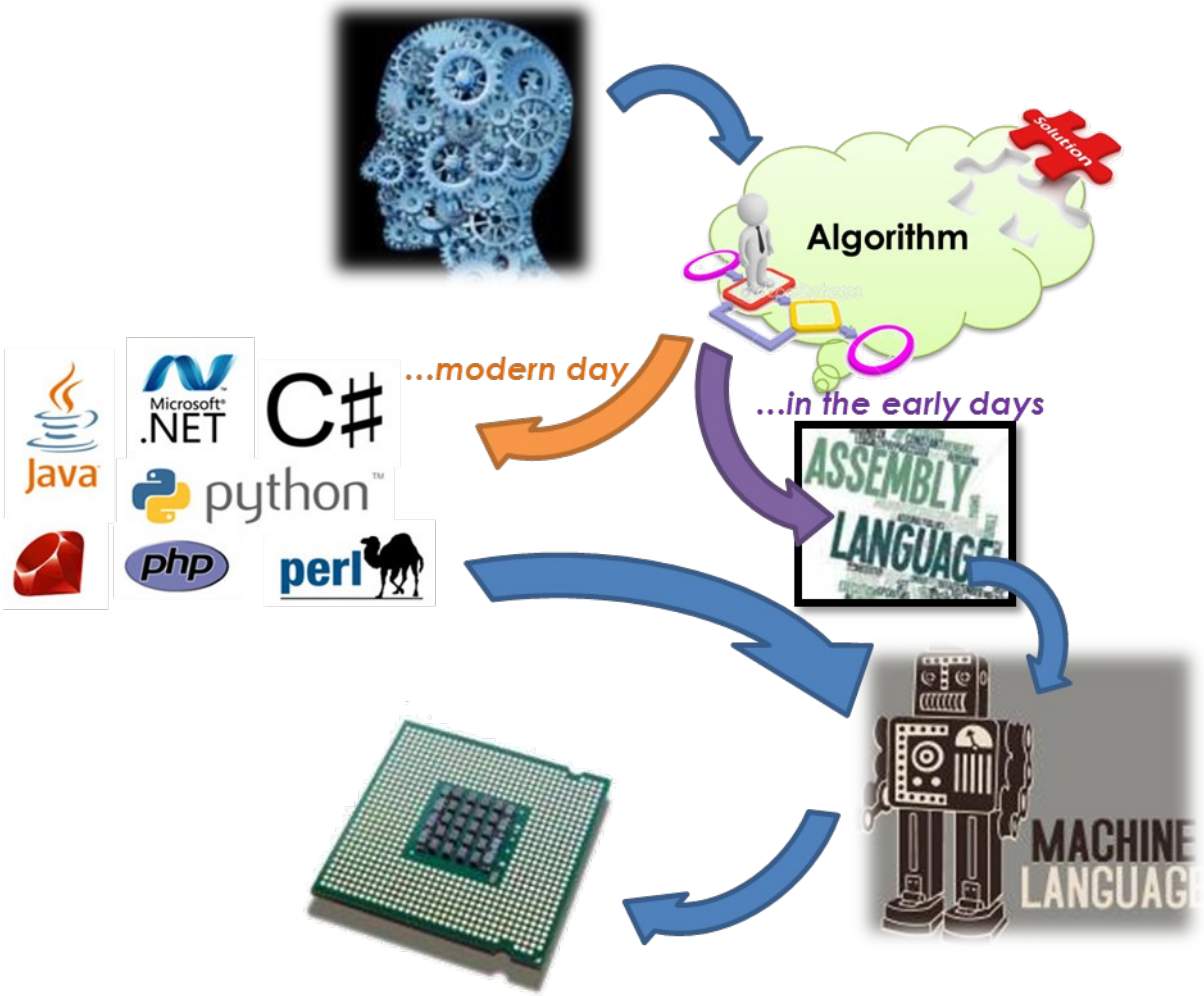
Translators and Facilities of Languages

Introduction

CPUs are very impressive but they are actually quite simple when it comes to processing. They can only process 1's and 0's. They therefore do not understand how to process programming code in the form in which we write it (e.g. Python). So what has to happen to get our code into a form that the CPU can work with...?

From Our Brain to the CPU...



Machine Code (Very Low Level Language)

The CPU can only understand one language, "Machine Code". It is made up of coded instructions and data e.g. 01001100 (binary) or AB200 (hexadecimal). Machine code is stored as a binary number and then decoded and executed by the CPU's logic circuits. This means that writing programs in 'Machine Code' is difficult for a human to do.

Assembly Language (Low Level Language)

The fact that it was so difficult for early programmers to remember the binary code for a great number of different instructions, assembly language was created. This language has a small set of commands which represent certain pieces of machine code. This helped programmers as they didn't have to remember sets of binary code, instead they learnt commands. EG: Instead of memorising what 1011 or 1001 meant, they just had to remember commands like ADD and SUB. These commands are known as Mnemonics (simple memory

Computer Science UK Membership Site Licence: Do not share outside your centre.

aids). Assembly language is still quite difficult to learn and use and this is why High Level Languages were produced. Having said that, programmers do still use assembly language as it can be processed by the CPU far more quickly than high level languages. If a programmer wants greater control over what the CPU executes in order to run a small program, assembly language may be a better option.

High Level Languages (Python, Java, C++ etc)

High level code (aka 'source code') is far more easy to write and therefore for humans to understand. Its purpose is to be easier to write AND STILL be easily translated into machine code so that it can be processed by the CPU. To help it be translated, it makes use of KEY WORDS and SYNTAX *(rules for the keywords and arguments that go with them)*.

Using keywords, as opposed to any old English means that translations can happen – reserved words such as PRINT are known to translators whereas 'put on screen' is not etc. A translator can therefore convert particular instructions (that we write in a high level language) into machine code, so that the CPU can execute them.

High Level Language Translators

Translators are programs that convert high level language commands, such as PRINT, IF and FOR, into a set of machine code commands such as 1011, 11001 and 11000011110, so that the CPU can process the data.

There are 2 ways in which translators work:

- Take the whole code and convert it into machine code before running it (known as compiling).
- Take the code one instruction at a time, translate and run the instruction, before translating the next instruction (known as interpreting).

Compiler	Interpreter
Converts the whole code into one file (often an .exe file). The file can then be run on any computer without the translator needing to be present. Can take a long time to compile source code as the translator will often have to convert the instructions into various sets of machine code as different CPUs will understand instructions with different machine code from one another.	Converts the source code into machine code 1 line at a time. Program therefore runs very slowly. Main reason why an interpreter is used is at the testing / development stage. Programmers can quickly identify errors and fix them. The translator must be present on the computer for the program to be run.

It must be noted that assembly language (low level language) has its own translator too. The translator is known as an assembler. It converts mnemonic assembly language instructions into machine code.

Computer Science UK Membership Site Licence: Do not share outside your centre.

Integrated Development Environment (IDE)

An Integrated Development Environment (IDE) is a software package. It provides the tools for a computer programmer to develop a software. It allows programmers to write their high level code, test it and translate it.

IDEs will often be written for specific high level languages (e.g. PyScripter is built for Python), but some support the writing of a number of different languages.

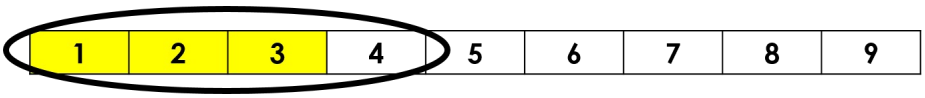
They will normally offer the following features:

- Source Code Editor – allowing the writing and editing of code.
- Interpreter – allows source code to be translated into machine code one line at a time for testing.
- Automation Tools – automate tasks such as finishing off key words and indenting on your behalf.
- Debugger – identifies logic and syntax errors and shows where they are.
- Compiler – converts entire source code into machine code so it can be run as its own individual program file.
- Auto-Documentation – stores lists of variables, modules, function calls etc, which are documented for other programmers.

Computer Science UK Membership Site Licence: Do not share outside your centre.

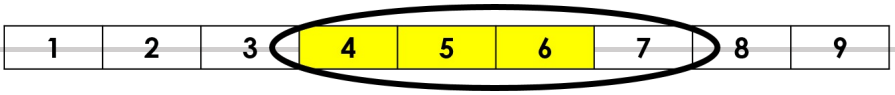
Questions *(The question zone you choose must either match your target grade or be higher!)*

Question Zone 1-3



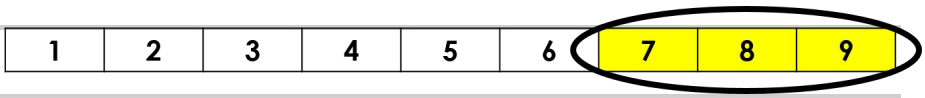
1. What does the term 'high level language' mean? [2]
2. What does a translator do and why is it required? [3]
3. Describe the purpose of an IDE and explain some of the various features of a typical IDE. [5]

Question Zone 4-6



1. Describe the purpose of an IDE and explain some of the various features of a typical IDE. [5]
2. Explain the difference between an interpreter and a compiler. [4]
3. Explain the difference between machine and assembly language? [4]

Questions Zone 7-9



1. Explain the difference between machine and assembly language? [4]
2. Explain the reasons that a programmer may have for writing software in assembly language, which is to be installed on an embedded system. [4]
3. Discuss the suitability of the different types of translator for a programmer during development and after development. [5]

Computer Science UK Membership Site Licence: Do not share outside your centre.

Checklist:

- Date and title, clearly presented
- Spelling & grammar checked
- Question numbers in the margin
- Handwriting neat & legible
- Punctuation / Capital letters

Keywords / Key Terms:

Compiler: A type of translator that converts high-level code into machine code, in one go, producing a standalone executable file.

Interpreter: A type of translator that converts high-level code into machine code, one line at a time, executing that line of code before translating and executing the next.

IDE*:* Integrated Development Environment – A type of program, which supports the development of software.

Computer Science UK Membership Site Licence: Do not share outside your centre.

Stick answer sheet here

State/Identify/Give/Name: Simply label a diagram, fill out a table or write a few words
Describe: Describing is 'saying what you see' (E.G.: A computer will have a CPU, Primary and Secondary storage etc)
Explain: Explaining is 'saying WHY/HOW something is like that'. (E.G.: A computer will have a CPU so that it can process all of the data the computer needs to perform a range of tasks. Primary and Secondary storage is needed because...)
Discuss: Discussing is 'looking at two sides of an issue, weighing up the two views and giving a conclusion'. Often these require a mini essay answer. (E.G.: New technology could be seen as being bad for the environment because..., but on the other hand, new technology has led to... In conclusion I believe that...)
Describe/Explain/Discuss **using examples**: Finally, if you are asked to give examples in any of these types of questions – YOU MUST **GIVE EXAMPLES**!

Computer Science UK Membership Site Licence: Do not share outside your centre.

Reflections **Score:** / **Percentage:** % **Grade:** **Progress** On / Above / Below

What Went

- ☐ I demonstrated a good level of understanding.
- ☐ I responded to the command words effectively.
- ☐ My answers were detailed / were written in depth.
- ☐ My work was well presented / legible.
- ☐ My answers effectively incorporated technical terminology.
- ☐ My responses were well structured / organised.
- ☐ My revision strategy was effective as I showed depth of understanding in my answers.
- ☐ My answers contained enough points / examples / explanations to achieve the marks available.

Even Better

- ☐ My answers need to be more accurate.
- ☐ I must respond correctly to the command words.
- ☐ My answers need more detail / greater depth.
- ☐ I must take greater care over my work / write neatly.
- ☐ I must incorporate key terminology into my answers.
- ☐ I must better organise my answers to improve its clarity.
- ☐ I need to improve my revision strategy, as I did not demonstrate a depth of understanding in my answers.
- ☐ My answers didn't contain enough points / examples / explanations to achieve the marks available.

Further thoughts:

Computer Science UK Membership Site Licence: Do not share outside your centre.